Виктор Крапивенский
Резюме

Обо мне

Программист. Мои интересы включают в себя алгоритмы и структуры данных, техники написания чистого и поддерживаемого кода, дизайн (проектирование) программного обеспечения, компияторы и интерпретаторы, многопоточное и многопроцессорное программирование, системное и низкоуровневое программирование, вычислительную теорию чисел. Автор нескольких проектов с открытым исходным кодом и научных статей.

Навыки

Проектирование программного обеспечения · Алгоритмы и структуры данных · C · C++ · Go · Python · Lua · JavaScript · ассемблер x86-64.

Опыт

2021—сейчас · **Программист (частная компания)** · Разработал поставщиков рыночных данных для нескольких криптобирж, программы для алгоритмической торговли на нескольких криптобиржах, программы для передачи рыночных данных по сети с низкой задержкой, и другие программы, на языке Си · Разработал быстрый парсер JSON на языке Си · Разработал эффективное параллельное вычисление цифровой подписи, основанной на Pedersen hash, вычисление которой требуется для торговли на криптобирже dYdX, на Си и ассемблере x86-64 · Разработал быстрый эмулятор для программ на EVM (Ethereum Virtual Machine), нужный для расчёта проскальзывания цены для заданного объёма для децентрализованных обменников (Decentralized Exchange, DEX) SushiSwap, Uniswap v2 и Uniswap v3, на Си и ассемблере x86-86.

2020 · **Разработчик Go** (контракт с **Offscale**) · Разработал **goffkv** (**goffkv-consul**, **goffkv-zk**, **goffkv-etcd**) — версия библиотеки **liboffkv** на языке Go. Язык Go имеет очень большие накладные расходы на FFI с родным ABI платформы, поэтому в сообществе Go принято «переписывать на Go» вместо использования каких-либо обвязок («биндингов») для взаимодействия с кодом на других языках (в том числе Си и C++, на котором написана liboffkv).

2019 · **Программист Rust** (контракт с **Fantom foundation**) · Разрабатывал утилиты для внутреннего использования.

2019 · **Архитектор программного обеспечения** (контракт с **Sikoba Research**) · Внедрил поддержку LLVM во фреймворке проверяемых вычислений (verifiable computation) **isekai**, на языке Crystal. См. следующие статьи для получения дополнительной информации:

- **Isekai LLVM update #1**;
- **Isekai LLVM update #2: conditionals and loops**;
- **Isekai LLVM: final update**.

2019 · **Разработчик C++** (контракт с **Offscale**) · Разработал **liboffkv**, единообразный интерфейс для распределённых key-value хранилищ, в команде из четырёх человек; сделал обвязки («биндинги») для яызка Си; сделал вклад в проект **ppconsul**: поддержка транзакций.

2018 · **Программист (частная компания)** · Разрабатывал торговых роботов и различные утилиты для анализа потока заявок и торговли на нескольких криптобиржах (используемые языки: Python, C++).

2017 · **Стажёр** (Google Summer of Code) · Разработал поддержку Lua-скриптинга в проекте strace (используемые языки: Си, Lua).

Проекты

2016—сейчас **luastatus**, универсальный генератор наполнения для строк состояния

luastatus — универсальный генератор наполнения для строк состояния. Он позволяет пользователю конфигурировать, как обрабатывать и отображать данных из различных источников событий, с помощью языка Lua. Главная особенность заключается в том, что содержимое может обновляться мгновенно при возникновении какого-либо события, будь то изменение раскладки клавиатуры, громкости звука в системе, заголовка активного окна или песни в вашем любимом музыкальном проигрывателе (при условии наличия соответствующего плагина), что довольно редко встречается в тайлинговых оконных менеджерах. Его девиз:

> Больше никаких часто форкающихся, запаздывающих на секунду, основанных на shell-скриптах генераторов строк состояния!

Он имеет модульную архитектуру, которая поддерживает плагины для предоставления данных из различных источников и библиотеки (barlibs) для взаимодействия с различными строками состояния. Он поддерживает *i3wm*, *dwm*, *lemonbar*, *dzen*/*dzen2*, *xmobar*, *yabar*, *dvtm* и другие.

2017 **Поддержка Lua-скриптинга в strace**, проект в рамках Google Summer of Code—2017

Я расширил возможности проекта *strace*, добавив функцию вмешательства в отслеживаемый процесс, позволяющую пользователю внедрять фальшивые результаты системных вызовов, а также читать и записывать данные в память отслеживаемого процесса, на языке Lua.

2020 **libdeci**, библиотека арифметики десятичных чисел произвольной точности для языка Си

Это библиотека для Си, предназначенная для работы с десятичными числами произвольной точности, с дополнительными библиотеками **libdeci-kara**, которая реализует алгоритм Карацубы для умножения, **libdeci-ntt** которая реализует умножение с помощью теоретико-числового преобразования (number-theoretic transform, NTT), варианта быстрого преобразования Фурье, **libdeci-newt**, которая реализует быстрое деление и получение приближения обратного с помощью метода Ньютона. Она быстрее, чем библиотека *mpdecimal* (использующая, например, в *Python*).

2020—сейчас **calx**, bc-подобный язык программирования

calx — это попытка сделать современную замену *bc*, при этом сохраняя его лучшие свойства, такие как десятичная арифметика произвольной точности и явная поддержка интерактивности в языке. Это полноценный язык программирования с функциями (и функциями высшего порядка), локальными и глобальными переменными, списками, словарями, строками и т.д.

2020 **“Speeding up decimal multiplication”**, исследовательский проект (**URL на ArXiv.org**).

В рамках данного исследовательского проекта достигнуто ускорение в 3—5 раз по сравнению с библиотекой *mpdecimal*. В статье описывается реализация и обсуждаются дальнейшие возможные оптимизации. Также представлен простой, эффективный с точки зрения кэширования алгоритм для транспонирования матриц $2^n \times n$ или $n \times 2^n$ на месте, необходимость в котором возникает в варианте «шестишагового алгоритма» матричного алгоритма Фурье, про который, по-видимому, нет широкой осведомлённости. Ещё одна находка заключается в том, что использование двух простых модулей вместо трёх имеет смысл даже с учетом наихудшего случая увеличения размера входных данных и упрощает восстановление ответа.

2022 **FiWiA**, генератор машинного кода x86-64 для арифметических операций над числами шириной в фиксированное количество машинных слов

Необходимость в арифметике над числами размера в фиксированное число машинных слов часто возникает в криптографии. В этих условиях, полная размотка (unrolling) цикла обычно желательна по двум причинам: накладные расходы на счётчик цикла и необходимость передавать флаг переноса в следующую итерацию, что, без размотки цикла, необходимо было бы делать с помощью сохранения флага переноса в регистр с последующим его восстановлением из регистра, что плохо сказывается на производительности. *FiWiA* генерирует машинный код для x86-64 с полностью размотанными циклами для арифметических операций над числами размера в фиксированное число машинных слов, таких как сложение/вычитание, маскированное сложение/вычитание, отрицание, сравнение, умножение, битовые сдвиги.

2022 **sloppy-json**, span-ориентированный парсер JSON для языка Си, с внешним препроцессором для упрощения процесса парсинга

Эргономичный парсинг JSON на языке Си — довольно сложная задача: практически все существующие парсеры либо:

- DOM-style (напр. *cJSON*): они преобразуют JSON в «объекты», используя **динамически выделяемые** массивы и словари, из которых пользователь должен «достать» нужные данные, после чего уничтожить эти «объекты»; или
- SAX-style (напр. *YAJL*): они позволяют пользователю итерироваться по токенам JSON, что на практике означает частое использование функций обратного вызова (callbacks), которые крайне утомительно писать и которые приводят к значительной потере производительности, поскольку функции обратного вызова вызываются по указателю и не могут быть встроены (inlined).

В ситуациях, когда мы не можем себе позволить динамическое выделение, например, когда важна задержка в худшем случае, DOM-style парсеры использовать невозможно. Вместо того, чтобы фокусироваться на объектах JSON, *sloppy-json* работает со span'ами (span — это пара из указателя и длины), которые, если их распарсить, представляют собой объекты JSON. Он предоставляет функции для классификации span'ов, итерации по span'ам, представляющим массивы и словари, преобразования строк в числа, разэкранирования строк и остальных вещей, которые необходимы для «парсинга» JSON. Он не выделяет память. Он также поставляется с внешним препроцессором, написанным на Python, для «подслащения» (as in «синтаксический сахар») процесса парсинга. Он также предоставляет некоторые вспомогательные функции для генерации JSON (например, форматирование чисел и экранирование строк).

2025 **"glass: ordered set data structure for client-side order books"** (**URL на ArXiv.org**), статья

В этой статье я представляю свою реализацию упорядоченного множества, которая предназначена для хранения биржевых книг заявок («стаканов»), основанную на префиксном дереве. Она поддерживает только целочисленные ключи и оптимизирована для рыночных данных, а именно для того, что в статье называется «последовательной локальностью». Я считаю новшествами следующие вещи:

- Использование *кэшированного пути* для извлечения преимущества из *последовательной локальности* в рыночных данных и быстрое его усечения при операции удаления;
- Хеш-таблица (или, скорее, *кэш-таблица*) с жёсткими $O(1)$ гарантиями по времени работы на все операции; она нужна для ускорения поиска ключа (вплоть до пре-листового узла префиксного дерева);
- Аппаратное ускорение операций поиска следующего/предыдущего установленного бита с помощью расширения набора инструкций **BMI2** на x86-64;
- *Фичи*, специфичные для книги заявок: принцип вытеснения и операция реструктуризации префиксного дерева, которые предотвращают чрезмерное потребление памяти деревом.

По сравнению со стандартным контейнером C++ *std::map* достигается следующее ускорение: в 6—20 раз на операциях модификации, в 30 раз на операциях поиска, в 9—15 раз на обработке реальных рыночных данных и более скромное ускорение в 2—3 раза на итерации.

2025—сейчас **jjhash**, простая строковая хеш-функция, обладающая лучшей скоростью и статистическими характеристиками, чем FNV-2

jjhash — это простая строковая хеш-функция. Она:

- быстрее, чем FNV-2 (в 6 раз быстрее на строках с заранее известной длиной, в 3 раза быстрее на нуль-терминированных строках);
- превосходит FNV-2 по статистическим свойствам, особенно для больших хеш-таблиц размером в степень двойки;
- поддерживает потоковое вычисление (т.е. быстрое вычисление хеша конкатенации $A + B$ на основе содержимого B и состояния хеширования A);
- имеет 32-битные и 64-битные варианты.

Награды

2016	Призёр всероссийской олимпиады школьников по информатике
2016	Победитель индивидуальной олимпиады школьников по информатике и программированию (ИОИП)
2017	4ое место в хакатоне “LAToken hackathon”
2018	1ое место в хакатоне “Global Changers”
2018	1ое место в хакатоне “IDACB & CryptoBazar hackathon”
2018	1ое место в хакатоне “Phystech.Genesis”
2018	3^{ье} место в хакатоне “CryptoBazar Serial Hacking: October”
2018	1ое место в хакатоне “CryptoBazar Serial Hacking: November”
2018	Менторство 2-х команд в хакатоне “CryptoBazar Serial Hacking: December”, занявших 2ые—3^{ьи} места
2019	1ое место в хакатоне “CryptoBazar Serial Hacking: Grand Finale”
2020	2ое место в хакатоне “VirusHack”

Контакты

shdownnine@gmail.com
https://github.com/shdown
https://www.linkedin.com/in/shdownnine